New Horizons®
Centro de Capacitación

# Curso

## Desarrollo Full Stack con C# y Angular

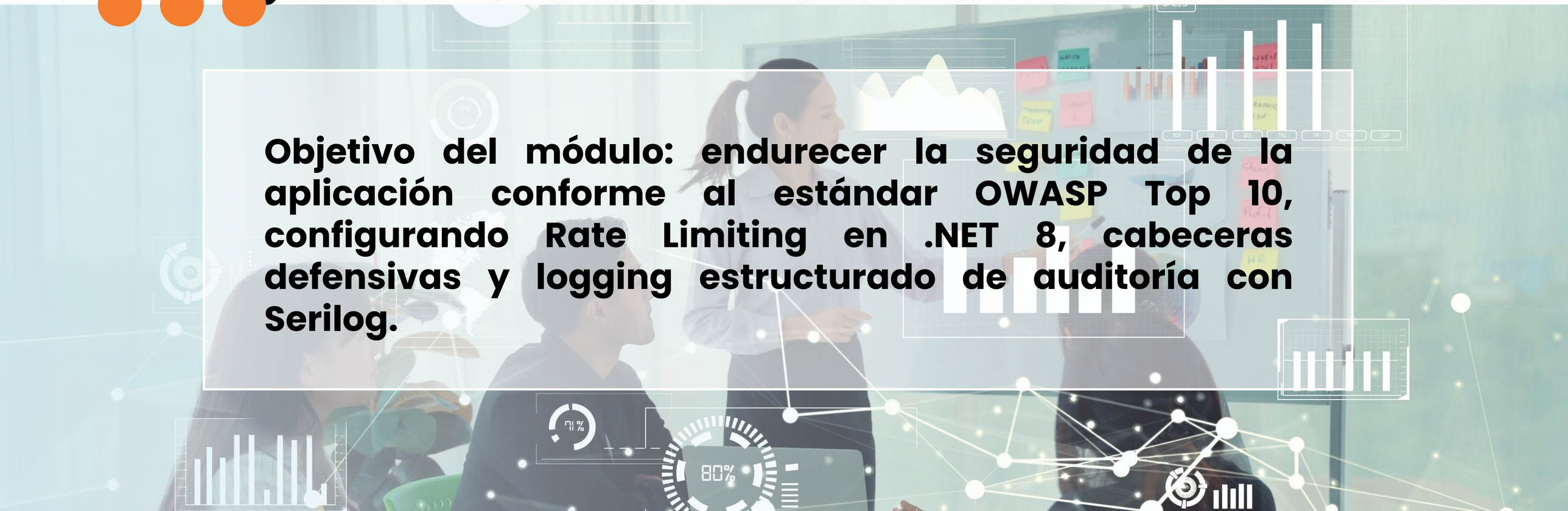
Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

# Sesión 16 — Módulo 4:

## Seguridad Web OWASP, Rate Limiting y Logging con Serilog



**Objetivo del módulo: endurecer la seguridad de la aplicación conforme al estándar OWASP Top 10, configurando Rate Limiting en .NET 8, cabeceras defensivas y logging estructurado de auditoría con Serilog.**




New Horizons®
Centro de Capacitación

# Agenda

- Vulnerabilidades OWASP Top 10 aplicadas a APIs REST y SPAs
- Prevención de inyección SQL, XSS, CSRF y exposición de datos sensibles
- Rate Limiting nativo en ASP.NET Core 8 para mitigación de ataques DoS y fuerza bruta
- Cabeceras de seguridad HTTP: CSP, HSTS, X-Frame-Options y X-Content-Type-Options
- Logging estructurado con Serilog: CorrelationId, sinks a consola y archivo rotativo




New Horizons®
Centro de Capacitación

# Sesión 1:

# Seguridad Web OWASP Top 10, Rate Limiting y Logging con Serilog

# Visión general de la sesión

New Horizons®
Centro de Capacitación



**Tu API es un banco.** Hoy le ponemos cerraduras contra los ladrones conocidos (OWASP), un torniquete en la puerta (Rate Limiting), un reglamento para el guardia del navegador (cabeceras) y cámaras que graban todo (Serilog).



## OWASP Top 10

*≈ Las cerraduras*

Inyección SQL, XSS, CSRF y fuga de datos sensibles.



## Rate Limiting

*≈ El torniquete*

Frena la fuerza bruta y el abuso sobre /api/auth/login.



## Cabeceras HTTP

*≈ El reglamento*

CSP, HSTS, X-Frame-Options y nosniff instruyen al navegador.



## Serilog

*≈ Las cámaras*

Logs estructurados con CorrelationId para auditoría.

# Pilares técnicos fundamentales

New Horizons
Centro de Capacitación

1 

**OWASP Top 10 en APIs y SPAs**

*≈ El cartel de los más buscados*

**Responde:** ¿Qué ataques son los más frecuentes?

2 

**Inyección SQL, XSS, CSRF y datos sensibles**

*≈ Las trampas en el formulario*

**Responde:** ¿Cómo convierte un atacante un dato en una orden?

3 

**Rate Limiting nativo en .NET 8**

*≈ El torniquete del metro*

**Responde:** ¿Cuántas veces puede tocar la puerta un mismo cliente?

4 

**Cabeceras HTTP: CSP, HSTS, X-Frame, nosniff**

*≈ El reglamento del guardia*

**Responde:** ¿Qué le prohibimos hacer al navegador?

5 

**Serilog: CorrelationId y archivo rotativo**

*≈ Las cámaras de seguridad*

**Responde:** ¿Qué pasó, a quién y en qué petición?

# OWASP Top 10: los más buscados

New Horizons®
Centro de Capacitación

🕵️ **El cartel de “Se busca”.** OWASP publica la lista de los 10 riesgos que más víctimas cobran en aplicaciones web. Así aparecen en la edición 2021 y así los cubrimos en el curso:

### A01
**Control de acceso roto**

S15 · roles y guards

### A02
**Fallas criptográficas**

S13 · PBKDF2 + HSTS

### A03
**Inyección (SQL, XSS)**

HOY · EF Core + Angular

### A04
**Diseño inseguro**

Curso · Clean Architecture

### A05
**Configuración insegura**

HOY · cabeceras HTTP

### A06
**Componentes vulnerables**

npm audit · dotnet list package --vulnerable

### A07
**Fallas de autenticación**

HOY · Rate Limiting en login

### A08
**Integridad de software y datos**

S19 · pipeline CI/CD

### A09
**Fallas de logging y monitoreo**

HOY · Serilog

### A10
**SSRF**

Validar URLs salientes

# Práctica: Rate Limiting y cabeceras HTTP

New Horizons
Centro de Capacitación

**La receta del cerrajero:** primero decides cuántas llaves das, luego en qué puerta instalas la cerradura, después pegas el reglamento y al final pruebas forzarla.

1

## Definir la cuota

AddRateLimiter con política “login”: 5 peticiones por minuto por IP.

*→ Lámina 13*

2

## Aplicarla al login

RequireRateLimiting("login") o [EnableRateLimiting] en el endpoint.

*→ Lámina 13*

3

## Pegar las cabeceras

Middleware con CSP, X-Frame-Options, nosniff y UseHsts().

*→ Lámina 15*

4

## Compilar y forzar

Build en limpio y 6 logins seguidos: el 6.º debe devolver 429.

*→ Lámina 21*

✅ **Resultado esperado:** cinco intentos con `401 Unauthorized` y el sexto con `429 Too Many Requests`, más las cabeceras visibles en DevTools → Network → Headers.

# SQL Injection y EF Core

New Horizons
Centro de Capacitación

**SolicitudesRepository.cs**

```
// ❌ Vulnerable: concatenas el texto del usuario en el SQL
1 var sql = "SELECT * FROM Solicitudes WHERE Motivo LIKE '%"
          + filtro + "%'";
  var malas = db.Solicitudes.FromSqlRaw(sql).ToList();

// ✅ Seguro: LINQ genera parámetros (@p0) automáticamente
2 var buenas = await db.Solicitudes
      .Where(s => s.Motivo.Contains(filtro))
      .ToListAsync();

// ✅ Seguro: FromSql con interpolación = consulta parametrizada
3 var porCentro = await db.Solicitudes
      .FromSql($"SELECT * FROM Solicitudes WHERE CentroCostoId = {id}")
      .ToListAsync();
```



## La casilla trampa

El atacante no escribe un dato en la casilla: escribe una orden para el cajero.

**Si filtro = x' OR 1=1 --**

```
... WHERE Motivo LIKE '%x'
    OR 1=1 --%'
→ devuelve TODAS las filas
```

1 **Concatenar = abrir la puerta**

El texto del usuario se vuelve parte del SQL.

2 **LINQ = parámetros**

EF Core envía el valor como @p0, nunca como código.

3 **FromSql($"...") también**

Cada {valor} se convierte en DbParameter.

# XSS: Angular como escudo

New Horizons
Centro de Capacitación

**detalle.component.html / .ts**

```
<!-- ✅ Interpolación: Angular escapa el HTML automáticamente -->
<p>{{ solicitud.motivo }}</p>

<!-- ⚠️ innerHTML: Angular sanitiza (quita script, onerror...) -->
<div [innerHTML]="solicitud.descripcionHtml"></div>

<!-- ❌ bypassSecurityTrust* desactiva el escudo -->
<div [innerHTML]="htmlConfiado"></div>

// detalle.component.ts
// Nunca con datos que escribió un usuario:
this.htmlConfiado = this.sanitizer.bypassSecurityTrustHtml(entrada);
```

1
2
3

## Grafiti detrás de un vidrio

Angular pone un vidrio sobre la pared: el grafiti se ve, pero no puede “salir” y ejecutarse.

**Entrada: `<img src=x onerror=alert(1)>`**

| | |
|---|---|
| ✅ `{{ }}` | Se muestra como texto literal |
| ✅ `[innerHTML]` | Angular elimina el onerror |
| ⛔ `bypassSecurityTrust` | El script se ejecuta |

**Segunda capa:** la cabecera CSP (lámina 14) bloquea scripts inyectados aunque alguien se salte el sanitizador.

# CSRF y exposición de datos sensibles

New Horizons®
Centro de Capacitación

**UsuariosController.cs**

```
// ❌ Devuelve la entidad completa (¡incluye PasswordHash!)
[HttpGet("{id}")]
1 public async Task<Usuario?> Get(int id) =>
      await _db.Usuarios.FindAsync(id);

// ✅ Proyecta a un DTO con lo mínimo necesario
2 public record UsuarioDto(int Id, string Nombre, string Rol);

[HttpGet("{id}")]
public async Task<UsuarioDto?> Get(int id) =>
    await _db.Usuarios
3       .Where(u => u.Id == id)
        .Select(u => new UsuarioDto(u.Id, u.Nombre, u.Rol))
        .FirstOrDefaultAsync();
```

### La ventana sin cortina

Devolver la entidad entera es mostrar toda la casa a quien solo pidió ver la sala.

❌
```
{ "id": 7, "nombre": "Ana Demo",
"rol": "Gerente",
"passwordHash": "AQAAAAEAAC…" }
```

✅
```
{ "id": 7, "nombre": "Ana Demo",
"rol": "Gerente" }
```

### CSRF: el falsificador de firmas

Otra web hace que tu navegador “firme” una petición por ti.
Con el JWT en el header `Authorization` el navegador no lo adjunta solo → riesgo mínimo.
Si usas cookies: `SameSite=Strict` + antiforgery token.

# Rate Limiting: ¿qué algoritmo elegir?

New Horizons
Centro de Capacitación

**Todo local con aforo necesita un portero.** .NET 8 trae cuatro porteros distintos en Microsoft.AspNetCore.RateLimiting; la pregunta es cuál contratar para cada puerta.

## Fixed Window

***El torniquete que se reinicia cada minuto***

N peticiones por ventana fija. Simple; permite una ráfaga justo en el borde.

**Ideal:** Login, OTP

## Sliding Window

***La ventana que avanza con el reloj***

Divide la ventana en segmentos y suaviza el efecto borde.

**Ideal:** Login con mucho tráfico

## Token Bucket

***La alcancía de fichas***

Cada petición gasta una ficha; se rellenan a ritmo constante. Admite ráfagas controladas.

**Ideal:** APIs públicas

## Concurrency

***El aforo de la discoteca***

Limita peticiones simultáneas, no por tiempo.

**Ideal:** Reportes y exportaciones pesadas

🎯 **Para /api/auth/login:** Fixed o Sliding Window particionado por IP (y si puedes, también por usuario). Token Bucket brilla en APIs públicas con ráfagas.

# Rate Limiting nativo en .NET 8

New Horizons
Centro de Capacitación

**Program.cs**

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
1   o.AddPolicy("login", ctx => RateLimitPartition.GetFixedWindowLimiter(
2       ctx.Connection.RemoteIpAddress?.ToString() ?? "anon",
        _ => new FixedWindowRateLimiterOptions {
            PermitLimit = 5,
            Window = TimeSpan.FromMinutes(1),
            QueueLimit = 0 }));
});

3   app.UseRateLimiter();

4   app.MapPost("/api/auth/login", Login).RequireRateLimiting("login");
// En un controlador: [EnableRateLimiting("login")]
```



## El torniquete del metro

Cinco pases por minuto por IP. El sexto se queda afuera con un 429.

| | | |
|---|---|---|
| #1 · 401 | #2 · 401 | #3 · 401 |
| #4 · 401 | #5 · 401 | #6 · 429 ⛔ |

**1 Política con nombre “login”**

Se configura una vez y se reutiliza.

**2 Partición por IP**

Cada IP tiene su propio contador.

**3 UseRateLimiter()**

Después de UseRouting si lo llamas explícito.

**4 RequireRateLimiting**

Solo el login paga la política estricta.

# Cabeceras de seguridad HTTP

New Horizons
Centro de Capacitación

**El reglamento que viaja en cada respuesta.** El navegador es un guardia obediente: si le das instrucciones claras en las cabeceras, bloquea los ataques por ti.

## Content-Security-Policy

***≈ La lista de invitados***

Solo carga scripts y estilos de orígenes autorizados. Frena XSS inyectado.

```
default-src 'self'
```

## Strict-Transport-Security

***≈ Siempre por la puerta blindada***

El navegador recuerda usar solo HTTPS, incluso si escriben http://.

```
max-age=31536000
```

## X-Frame-Options

***≈ Prohibido enmarcar***

Nadie puede meter tu SPA en un iframe invisible (clickjacking).

```
DENY
```

## X-Content-Type-Options

***≈ No abras el paquete adivinando***

Evita que el navegador “adivine” el MIME y ejecute un .txt como script.

```
nosniff
```

# Middleware de cabeceras y HSTS

New Horizons®
Centro de Capacitación

**Program.cs**

```
1  app.Use(async (ctx, next) =>
   {
       var h = ctx.Response.Headers;
       h["X-Content-Type-Options"] = "nosniff";
       h["X-Frame-Options"] = "DENY";
       h["Referrer-Policy"] = "no-referrer";
2      h["Content-Security-Policy"] =
           "default-src 'self'; frame-ancestors 'none'";
       await next();
   });

   if (!app.Environment.IsDevelopment())
   {
3      app.UseHsts();        // Strict-Transport-Security
   }
   app.UseHttpsRedirection();
```



## La etiqueta en cada paquete

Un solo middleware pega el reglamento en todas las respuestas que salen de la API.

**DevTools → Network → Response Headers**

| | |
|---|---|
| ✓ x-content-type-options | nosniff |
| ✓ x-frame-options | DENY |
| ✓ referrer-policy | no-referrer |
| ✓ content-security-policy | default-src 'self'… |
| ✓ strict-transport-security | max-age=31536000 |

1. **app.Use = se aplica a todo**
2. **CSP con frame-ancestors**
3. **HSTS solo fuera de desarrollo**

# Serilog: configuración y sinks

New Horizons
Centro de Capacitación

**Program.cs**

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
1   .MinimumLevel.Override("Microsoft.AspNetCore",
        LogEventLevel.Warning)
2   .Enrich.FromLogContext()
    .WriteTo.Console()
3   .WriteTo.File("logs/api-.log",
        rollingInterval: RollingInterval.Day,
        retainedFileCountLimit: 14)
    .CreateLogger();

builder.Host.UseSerilog();

4 app.UseSerilogRequestLogging();
```



## Cámaras de seguridad

No grabas “pasó algo”: grabas quién, qué, cuándo y en qué petición.



### Consola

Lo ves en vivo mientras desarrollas.



### Archivo rotativo

logs/api-20261026.log · uno por día · 14 días.

1. **Override de Microsoft.* · Menos ruido del framework.**
2. **Enrich.FromLogContext · Habilita CorrelationId (lámina 19).**
3. **Archivo con rotación diaria · No llena el disco.**
4. **UseSerilogRequestLogging · Una línea resumen por petición.**

# Logging estructurado y auditoría

New Horizons®
Centro de Capacitación

**SolicitudesService.cs**

```
// ❌ Interpolación: pierde la estructura (solo texto plano)
1 _logger.LogInformation($"Solicitud {id} aprobada por {usuario}");

// ✅ Plantilla: cada valor queda como propiedad consultable
2 _logger.LogInformation(
      "Solicitud {SolicitudId} aprobada por {Usuario}", id, usuario);

// ✅ Evento de seguridad para auditoría
3 _logger.LogWarning(
      "Login fallido para {Email} desde {Ip}",
      Enmascarar(email), ip);
```

## Ficha de archivo vs. servilleta

Una nota en servilleta solo se lee; una ficha con campos se busca, se filtra y se cuenta.

### Lo que guarda Serilog con la plantilla

```
{
  "Level": "Information",
  "MessageTemplate":
    "Solicitud {SolicitudId} aprobada…",
  "SolicitudId": 42,
  "Usuario": "gerente.demo" }
```

💡 Ahora puedes buscar `SolicitudId = 42` en todos los servidores. Con la interpolación ❶ solo tendrías texto. La regla de qué **no** registrar está en la siguiente lámina.

# Lo que nunca va en un log

New Horizons
Centro de Capacitación

🙈 **El log es un vidrio, no una caja fuerte:** lo leen desarrolladores, soporte y herramientas externas. Todo lo que escribas ahí, asume que alguien más lo verá.

### 🔑 Contraseñas

✗ `password=MiClave123`

✓ **Nunca, ni enmascaradas**

### 🎟️ Tokens y API keys

✗ `Bearer eyJhbGciOi…`

✓ **Solo los últimos 4 caracteres**

### 💳 Tarjetas y cuentas

✗ `4111111111111111`

✓ **4111 **** **** 1111**

### 🪪 Documentos de identidad

✗ `DNI 45678912`

✓ **DNI ****8912**

### 📧 Correos y teléfonos

✗ `ana.demo@empresa.test`

✓ **an***@empresa.test**

### 📦 Body completo del request

✗ `{ ...todo el JSON... }`

✓ **Solo IDs y metadatos**

# CorrelationId: trazabilidad de punta a punta

New Horizons®
Centro de Capacitación

**CorrelationIdMiddleware (Program.cs)**

```
app.Use(async (ctx, next) =>
{
    var cid = ctx.Request.Headers["X-Correlation-ID"].FirstOrDefault()
              ?? Guid.NewGuid().ToString("N");

    ctx.Response.Headers["X-Correlation-ID"] = cid;

    using (LogContext.PushProperty("CorrelationId", cid))
    {
        await next();
    }
});
```

1
2
3



## El número de guía del courier

El mismo número acompaña al paquete en cada escala: así sabes dónde se perdió.

### Mismo cid = misma petición

```
10:42:07 [a3f9c1] HTTP POST /api/solicitudes

10:42:07 [a3f9c1] Validación OK SolicitudId=42

10:42:08 [a3f9c1] SQL INSERT 35 ms
```

1. **Reusa el ID si ya viene**
   Angular puede enviarlo desde un interceptor.
2. **Lo devuelve en la respuesta**
   Soporte lo pide al usuario.
3. **PushProperty**
   Todos los logs de la petición lo llevan.

# Anatomía de un ataque bloqueado

New Horizons
Centro de Capacitación

**Película de atraco frustrado:** así se ven las capas trabajando juntas cuando un bot intenta adivinar contraseñas.

### 1. El bot ataca

Prueba contraseñas contra /api/auth/login.

### 2. El torniquete cuenta

5 intentos por minuto para esa IP.

### 3. 429 al sexto

Responde Too Many Requests + Retry-After.

### 4. Serilog lo graba

Warning con IP y CorrelationId.

### 5. Alguien reacciona

Alerta, bloqueo en WAF o revisión.

```
logs/api-20261026.log

10:51:03 [WRN]  Login fallido para an***@empresa.test desde 203.0.113.25  {CorrelationId: 7d1e}

10:51:04 [WRN]  Login fallido para an***@empresa.test desde 203.0.113.25  {CorrelationId: 9b02}

10:51:05 [WRN]  Rate limit excedido política=login ip=203.0.113.25  status=429  {CorrelationId: c4a8}
```

# Verifica tu blindaje

New Horizons
Centro de Capacitación

**simulacro.sh**

```
# 1) Fuerza bruta simulada: el 6.º intento debe dar 429
for i in {1..6}; do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST https://localhost:5001/api/auth/login \
    -H "Content-Type: application/json" \
    -d '{"email":"demo@empresa.test","password":"x"}'
done

# 2) Cabeceras de seguridad
curl -sI https://localhost:5001/api/solicitudes \
  | grep -iE "x-frame|x-content|content-security"

# 3) Logs estructurados del día
tail -f logs/api-20261026.log
```

## El simulacro de incendio

No esperas al incendio real para saber si funcionan las alarmas: las pruebas hoy.

**Salida esperada del paso ①**

| | | | | | |
|---|---|---|---|---|---|
| 401 | 401 | 401 | 401 | 401 | 429 |

**1 Bucle de 6 logins**

Cinco 401 y un 429: el torniquete funciona.

**2 curl -I**

Muestra solo las cabeceras de la respuesta.

**3 tail -f**

Ves los Warning en vivo con su CorrelationId.

# Errores comunes y cómo evitarlos

New Horizons®
Centro de Capacitación

| ✗ Evita esto | ✓ Haz esto |
| --- | --- |
| Omitir la validación estricta de parámetros en el servidor | Validar siempre los contratos DTO en cada llamada HTTP |
| Almacenar información sensible en repositorios públicos | Utilizar variables de entorno seguras en desarrollo y producción |
| Desacoplar la UI de las políticas de seguridad del backend | Derivar la UI de las mismas políticas y claims del backend |
| Registrar contraseñas, tokens o tarjetas en los logs | Enmascarar o excluir los datos sensibles del log |
| Concatenar texto del usuario en FromSqlRaw | Usar LINQ o FromSql($"...") parametrizado |

# Checklist de seguridad para producción

New Horizons®
Centro de Capacitación



**Checklist de pre-vuelo:** ningún piloto despega sin revisarla; ninguna API sale a producción sin pasar por aquí.

| | | |
|---|---|---|
| ✓ | **Rate Limiting estricto en /api/auth/login** | → *Corta la fuerza bruta.* |
| ✓ | **Cabeceras CSP, HSTS, X-Frame-Options y nosniff** | → *El navegador trabaja a tu favor.* |
| ✓ | **Cero SQL concatenado; solo LINQ o FromSql parametrizado** | → *Adiós inyección SQL.* |
| ✓ | **Endpoints que devuelven DTOs, nunca entidades** | → *Ningún PasswordHash viaja por la red.* |
| ✓ | **Serilog con CorrelationId y archivo rotativo** | → *Trazabilidad para auditoría.* |
| ✓ | **Secretos en variables de entorno o un gestor de secretos** | → *Nada sensible en el repositorio.* |

# Resumen de la Sesión 16

New Horizons
Centro de Capacitación

## Defensa en capas

Ninguna capa sola basta; juntas minimizan el impacto.

*≈ El banco*

## Inyección y XSS

EF Core parametriza; Angular escapa y sanitiza.

*≈ La casilla trampa*

## Rate Limiting

Detiene la adivinación de contraseñas en el login.

*≈ El torniquete*

## Cabeceras HTTP

El navegador bloquea inyección y frames.

*≈ El reglamento*

## Serilog

Trazabilidad estructurada para auditoría.

*≈ Las cámaras*

## CorrelationId

Une todos los logs de una misma petición.

*≈ El número de guía*

# PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Cómo previene EF Core los ataques de SQL Injection en comparación con consultas directas?
2. ¿Qué algoritmo de Rate Limiting (Fixed Window, Sliding Window, Token Bucket) es mejor para Login?
3. ¿Qué información sensible NUNCA debe registrarse en los logs de la aplicación?
4. ¿Por qué el CorrelationId es indispensable para rastrear peticiones en entornos distribuidos?
5. ¿Cómo evitamos que atacantes ejecuten ataques de clickjacking contra nuestra SPA?

New Horizons
Centro de Capacitación

# Conclusiones

## Defensa en profundidad

La seguridad por capas (Defense in Depth) minimiza el impacto ante fallas en un solo componente.

## Torniquete en el login

Rate Limiting detiene ataques automatizados de adivinación de contraseñas sobre /api/auth/login.

## El navegador como aliado

Las cabeceras de seguridad instruyen al navegador para bloquear ataques de inyección y frames.

## Cámaras encendidas

Serilog proporciona trazabilidad inmutable y estructurada para análisis forense y auditoría.

## Listo para producción

El sistema queda certificado contra los vectores de ataque más comunes en producción.

# GRACIAS